Package 'rENM.data'

April 10, 2026

Type Package

Title Data assembly tools for the rENM framework

Version 0.1.0.9000

Author John L. Schnase

Maintainer John L. Schnase <jschnase@mac.com>

ORCID 0000-0001-7473-3977

Description The rENM framework is a modular sysem for reconstucting and analyzing long-term ecological niche dynamics using time-indexed environmental data and species occurrence records. Its packages support climate-based niche modeling, temporal analysis of climatic suitability, and reproducible workflows for studying ecological change.

Depends R (>= 4.1.0)

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

URL https://github.com/jschnase/rENM.data

BugReports https://github.com/jschnase/rENM.data/issues

Imports rENM.core,
future,
future.apply,
sf,
stats,
terra,
tools,
utils

Contents

`find_occurrence_extent` *Find the spatial extent of a species' eBird occurrence data*

Description

Computes a centered percentile bounding box from occurrence files and writes the result to an extent.txt file for use in downstream rENM preprocessing and modeling workflows.

Usage

```
find_occurrence_extent(alpha_code, bbox_pct = 99)
```

Arguments

`alpha_code`	Character. Four-letter banding code for the species. Used to construct the run directory path.
`bbox_pct`	Numeric. Percent of points to include in the centered bounding box with (0 < bbox_pct <= 100). Values less than 100 discard symmetric tails along each axis. Default = 99.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: Occurrence files are expected in:

`<project_dir>/runs/<alpha_code>/ _occs/`

with filenames of the form:

`of-<year>.csv`

Each file must contain coordinate columns equivalent to longitude and latitude. Common aliases are automatically detected.

Methods: A centered percentile bounding box is computed by selecting the central `bbox_pct` percent of valid points along each axis and discarding symmetric tails. For example:

- bbox_pct = 95 uses the (2.5, 97.5) quantiles
- bbox_pct = 100 uses the full min and max range

All valid coordinates across all files are pooled prior to computing quantiles.

Coordinate validation: For each file, the function:

- Locates longitude and latitude columns using common aliases: longitude, lon, x and latitude, lat, y
- Converts values to numeric and discards invalid rows
- Retains only coordinates within: longitude in (-180, 180) latitude in (-90, 90)
- Removes zero-zero coordinates (0, 0)

Outputs: The resulting bounding box is written to:

`<project_dir>/runs/<alpha_code>/ _occs/extent.txt`

with the format:

- Upper-left: (,)
- Lower-right: (,)

A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

using the section title:

`Processing summary (find_occurrence_extent)`

Workflow context: This function is intended to be used after occurrence preprocessing steps such as:

- get_ebird_occurrences()
- remove_duplicate_occurrences()
- thin_occurrences() or thin_occurrences2()
- tidy_occurrences()

At this stage, the _occs directory should contain one or more `of-<year>.csv` files with species and coordinate columns.

Value

Invisibly returns a List with the following components:

- alpha_code: Character species code
- bbox:
  - xmin: Minimum longitude
  - xmax: Maximum longitude
  - ymin: Minimum latitude
  - ymax: Maximum latitude
- paths:
  - extent: File path to extent.txt
  - log: File path to the run log
- counts:
  - files: Number of occurrence files scanned
  - rows_read: Total rows read across all files
  - points_used: Number of valid coordinate rows used
  - bbox_pct: Percentile used for bounding box

Side effects:

- Writes extent.txt to the _occs directory
- Appends a processing summary to the run log
- Writes progress messages to the console

Examples

```
## Not run:
  res_full <- find_occurrence_extent("CASP")
  res_full$bbox
  res_full$paths$extent

  res_95 <- find_occurrence_extent("CASP", bbox_pct = 95)
  res_95$bbox

## End(Not run)
```

`find_range_extent` *Find the spatial extent of a species' USGS GAP range*

Description

Computes a bounding box from a species GAP range polygon and writes the result to extent.txt for use in rENM preprocessing workflows.

Usage

```
find_range_extent(alpha_code, pad_pct = 2)
```

Arguments

`alpha_code`	Character. Species alpha code matching ALPHA.CODE in the species CSV file.
`pad_pct`	Numeric. Percent padding applied to the bounding box. Positive values expand and negative values contract the box. Must be greater than -100. Default = 2.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: The species table is read from:

`<project_dir>/data/ _species.csv`

with fallback to:

`<project_dir>/data/ species.csv`

The row where `ALPHA.CODE == alpha_code` is located and the corresponding `GAP.RANGE` value is extracted.

The GAP range shapefile is then loaded from:

`<project_dir>/data/shapefiles/ <gap_range>/<gap_range>.shp`

Methods: The shapefile geometry is validated and its coordinate reference system (CRS) is checked. If necessary, the geometry is transformed to WGS84 (EPSG:4326).

A bounding box is computed in longitude and latitude degrees and then adjusted using a symmetric padding factor defined by `pad_pct`.

The padding is applied relative to the bounding box center. For example:

- pad_pct = 2 expands width and height by 2 percent

- pad_pct = -2 contracts width and height by 2 percent

Values less than or equal to -100 are invalid.

Outputs: The resulting bounding box is written to:

`<project_dir>/runs/<alpha_code>/ _occs/extent.txt`

Any existing extent.txt file is backed up prior to writing.

The output file contains:

- Upper-left: (,)
- Lower-right: (,)

Coordinates are written in decimal degrees with six decimal places of precision.

A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

using the section title:

`Processing summary (find_range_extent)`

Workflow context: This function is intended for workflows that rely on GAP range polygons to define spatial extents for downstream analyses. It can be used alongside occurrence-based preprocessing steps such as:

- get_ebird_occurrences()
- remove_duplicate_occurrences()
- thin_occurrences() or thin_occurrences2()
- tidy_occurrences()

Data requirements: The species CSV must contain:

- ALPHA.CODE: species identifier
- GAP.RANGE: shapefile directory name

The shapefile must exist at the expected path and contain valid geometry.

Value

List (invisibly returned) with elements:

- ul_lon: Numeric upper-left longitude
- ul_lat: Numeric upper-left latitude
- lr_lon: Numeric lower-right longitude
- lr_lat: Numeric lower-right latitude

Side effects:

- Writes extent.txt to the _occs directory
- Backs up any existing extent.txt file
- Appends a processing summary to the run log
- Writes progress messages to the console

Examples

```
## Not run:
  res <- find_range_extent("CASP", pad_pct = 2)
  res

  res_tight <- find_range_extent("CASP", pad_pct = -1)

## End(Not run)
```

`get_ebird_occurrences` *Read and process an eBird EBD occurrence record file*

Description

Extracts occurrence records for a single species from an eBird Basic Dataset (EBD) text file, filters valid coordinates, and organizes the data into temporal bins for rENM workflows.

Usage

```
get_ebird_occurrences(alpha_code, project_dir = NULL)
```

Arguments

`alpha_code`	Character. Four-letter species code.
`project_dir`	Character, NULL. Optional project root directory. If NULL, resolved using `rENM_project_dir()` via options or environment variables.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: The EBD text file is located at:

`<project_dir>/data/ebird/<ebd_dataset>/ <ebd_dataset>.txt`

where <ebd_dataset> is obtained from `get_species_info()` via the EBD.RECORDS field.

Methods: The function performs the following steps:

- Reads the EBD text file using streaming input
- Filters records to valid decimal-degree coordinates
- Writes a cleaned CSV copy to the _occs directory
- Parses observation dates into years
- Assigns records to 5-year bins spanning 1980 to 2024
- Writes one CSV file per bin to the _occs/tmp directory

Longitude and latitude columns are automatically detected using common aliases. Coordinates are converted to numeric and filtered to valid ranges:

- longitude in (-180, 180)
- latitude in (-90, 90)

- excludes (0, 0) coordinates

Duplicate records are retained.

Outputs: A cleaned CSV copy is written to:

`<project_dir>/runs/<alpha_code>/ _occs/<ebd_dataset>.csv`

Per-bin occurrence files are written to:

`<project_dir>/runs/<alpha_code>/ _occs/tmp/`

with filenames:

- of-.csv

A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

using the section title:

`Processing summary (get_ebird_occurrences)`

Data requirements: The EBD file must contain longitude, latitude, and date columns or recognizable aliases. At least one valid coordinate record must be present.

Value

Invisibly returns a List with the following components:

- csv_copy: File path to the cleaned CSV
- bin_dir: Directory containing per-bin CSV files
- bin_files: Character vector of per-bin file paths
- log_file: File path to the run log
- n_read: Integer number of rows read from the EBD file
- n_valid: Integer number of valid coordinate rows
- total_written: Integer total rows written across all bins

Side effects:

- Writes cleaned CSV and per-bin files to disk
- Creates directories as needed
- Appends a processing summary to the run log
- Writes progress messages to the console

Examples

```
## Not run:
  occ <- get_ebird_occurrences("CASP")
  utils::head(occ$csv_copy)

## End(Not run)
```

`get_merra_variables` *Assemble a run-tailored set of MERRA-2 variables for further processing*

Description

Crops and exports MERRA-2 variables into 5-year bins for a species, writing run-specific predictor rasters for downstream rENM modeling.

Usage

```
get_merra_variables(
  alpha_code,
  m2_vars = NULL,
  mc_vars = NULL,
  file_type = c(".tif", ".asc")
)
```

Arguments

`alpha_code`	Character. Four-letter species code used to locate the run directory and extent file.
`m2_vars`	Character, NULL. Optional vector of variable basenames to include from the m2 directory.
`mc_vars`	Character, NULL. Optional vector of variable basenames to include from the mc directory.
`file_type`	Character. Output formats to write. Allowed values are ".tif" and ".asc".

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: MERRA-2 rasters are read from:

`<project_dir>/data/merra/m2/<year>/ <project_dir>/data/merra/mc/<year>/`

where corresponds to 5-year bins (1980, 1985, ..., 2020).

Files may be in .tif, .tiff, or .asc format. If multiple files share the same basename, duplicates are resolved with preference for .tif.

The spatial extent is read from:

`<project_dir>/runs/<alpha_code>/ _occs/extent.txt`

which must contain "Upper-left" and "Lower-right" coordinates in (lon, lat) order.

Methods: For each 5-year bin, the function:

- Identifies candidate rasters from m2 and mc directories
- Filters by basename using m2_vars and mc_vars if provided
- De-duplicates files by basename with preference for GeoTIFF
- Crops rasters to the specified bounding box

Geographic rasters:

- Detects longitude range (0-360) and rotates to (-180, 180)
- Handles antimeridian crossing during cropping

Projected rasters:

- Builds a WGS84 bounding box polygon
- Projects the polygon into the raster CRS
- Crops using the projected geometry

Only rasters with valid cells after cropping are written.

Outputs: Cropped rasters are written to:

`<project_dir>/runs/<alpha_code>/ _vars/<year>/`

Output formats are controlled by `file_type`:

- .tif: GeoTIFF with CRS preserved or assigned (WGS84 if missing)
- .asc: ASCII grid; sidecar files (.aux.xml, .xml, .prj) are removed

Log file: A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

including:

- alpha code
- extent used (xmin, xmax, ymin, ymax)
- input m2 and mc roots
- output directory
- per-year processing results
- skipped files and reasons

Data requirements: The extent.txt file must exist and be correctly formatted. Input raster directories must contain at least one valid raster file.

Value

Invisibly returns a Data frame summarizing processing results:

- year: Integer bin year
- family: Character indicating m2 or mc source
- basename: Character variable name
- in_file: Character input file path
- tif_written: Logical indicating GeoTIFF output success
- asc_written: Logical indicating ASCII output success
- notes: Character processing notes or failure reason

Side effects:

- Writes cropped raster files to disk
- Creates output directories as needed
- Removes ASCII sidecar files when applicable
- Appends a processing summary to the run log
- Writes progress messages to the console

Examples

```
## Not run:
  get_merra_variables("CASP")

## End(Not run)
```

`limit_record_count` *Limit occurrence records per bin by random sampling*

Description

Randomly downsamples occurrence records within each per-bin CSV file to a maximum of `record_count` rows while preserving all records when counts fall below the specified threshold.

Usage

```
limit_record_count(alpha_code, record_count, project_dir = NULL)
```

Arguments

`alpha_code`	Character. Four-letter species alpha code (e.g., `"CASP"`).
`record_count`	Integer. Maximum number of records to retain per bin. Must be positive.
`project_dir`	Character, NULL. Optional project root directory. If NULL, resolved via `rENM_project_dir()`.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: This function operates on occurrence files located in:

`<project_dir>/runs/<alpha_code>/ _occs/tmp/`

Each file follows the naming pattern:

`of-<year>.csv`

Methods:

- Reads each `of-<year>.csv` file
- Counts the number of rows
- Randomly samples up to `record_count` rows without replacement
- Writes the sampled dataset back to the same file

Sampling uses base R `sample()`, allowing reproducibility when a random seed is set externally (e.g., `set.seed()`).

Outputs: Each input file is overwritten with the sampled dataset in:

`<project_dir>/runs/<alpha_code>/ _occs/tmp/`

Log file: A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

including total counts before and after sampling and per-file summaries.

Typical use cases:

- Balancing sample sizes across temporal bins
- Sensitivity analysis of model performance vs. sample size
- Reducing computational load for downstream modeling

Value

List (invisibly returned) with elements:

- `alpha_code`: Character species alpha code
- `files_processed`: Character vector of processed file paths
- `counts_before`: Named integer vector of original counts
- `counts_after`: Named integer vector of retained counts
- `output_dir`: Character directory containing processed files
- `log_file`: Character path to the run log file

Side effects:

- Overwrites occurrence CSV files in the tmp directory
- Appends a processing summary to the run log
- Writes progress messages to the console

See Also

`get_ebird_occurrences()`, `remove_duplicate_occurrences()`, `thin_occurrences()`, `tidy_occurrences()`

Other occurrence processing: `remove_duplicate_occurrences()`

Examples

```
## Not run:
  limit_record_count("CASP", record_count = 1000)

  limit_record_count(
    "CASP",
    record_count = 500,
    project_dir = "/projects/rENM"
  )

## End(Not run)
```

`remove_duplicate_occurrences` *Remove duplicate occurrence records from EBD occurrence datasets*

Description

Removes duplicate spatial records (longitude/latitude pairs) from per-bin occurrence CSV files generated during earlier rENM preprocessing steps.

Usage

```
remove_duplicate_occurrences(alpha_code, project_dir = NULL)
```

Arguments

alpha_code	Character(1). Four-letter species alpha code (e.g., "CASP").
project_dir	Character(1) or NULL. Optional project root directory. If NULL, resolved via rENM_project_dir().

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: Occurrence files are read from:

<project_dir>/runs/<alpha_code>/ _occs/tmp/

Each file is expected to follow the naming pattern:

- of-<year>.csv

and contain columns named longitude and latitude.

Methods:

- Reads each per-bin CSV file
- Identifies duplicate rows based on identical longitude and latitude
- Retains the first occurrence of each unique coordinate pair
- Overwrites the original file with the de-duplicated dataset

Duplicate records are defined strictly as rows sharing identical longitude and latitude values.

Outputs: Cleaned occurrence files are written back to:

<project_dir>/runs/<alpha_code>/ _occs/tmp/

replacing the original files.

A processing summary is appended to:

<project_dir>/runs/<alpha_code>/ _log.txt

under the section:

Processing summary (remove_duplicate_occurrences)

Workflow context: This function is typically used after temporal binning and prior to spatial thinning or modeling steps. It assumes that occurrence files have been created by get_ebird_occurrences() or equivalent preprocessing steps.

Value

Invisibly returns a list with:

- alpha_code: Species alpha code
- files_processed: Character vector of processed file paths
- duplicates_removed: Named integer vector of counts removed per file
- output_dir: Directory containing cleaned occurrence files
- log_file: Path to the run log file

See Also

Other occurrence processing: `limit_record_count()`

Examples

```
## Not run:
  remove_duplicate_occurrences("CASP")

  remove_duplicate_occurrences(
    "CASP",
    project_dir = "/projects/rENM"
  )

## End(Not run)
```

`set_extent` *Set the spatial extent to be used in further model processing*

Description

Create or update an `extent.txt` file for a species run, preserving the format expected by downstream functions.

Usage

```
set_extent(alpha_code, ul = c(-128, 49), lr = c(-66.5, 24))
```

Arguments

`alpha_code`	Character(1). Species alpha code used to locate the run directory under `<project_dir>/runs/<alpha`
`ul`	Numeric(2). Upper-left corner (lon, lat) of the bounding box. Default `c(-128.0, 49.0)`.
`lr`	Numeric(2). Lower-right corner (lon, lat) of the bounding box. Default `c(-66.5, 24.0)`.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: The extent file is written to:

`<project_dir>/runs/<alpha_code>/ _occs/extent.txt`

Coordinates are provided as:

- `ul`: upper-left (lon, lat)
- `lr`: lower-right (lon, lat)

Methods:

- Validates coordinate structure and numeric ranges
- Checks for canonical orientation of bounding box corners

- Renames any existing `extent.txt` to a backup file
- Writes a new `extent.txt` using standardized formatting
- Appends a processing summary to the run log

Coordinates must satisfy:

- longitude in (-180, 180)
- latitude in (-90, 90)

Orientation checks:

- Upper-left latitude should exceed lower-right latitude
- Upper-left longitude should be less than lower-right longitude

Non-canonical orientation is reported but not corrected.

Outputs: The `extent.txt` file contains:

- Header metadata (timestamp, source, coordinate order)
- Upper-left and lower-right coordinates in (lon, lat) format

Existing files are backed up as:

`extent.txt.original`

or a timestamped variant if a backup already exists.

A processing summary is appended to:

`<project_dir>/runs/<alpha_code>/ _log.txt`

Data requirements: Coordinates must be numeric length-2 vectors with finite values.

Value

Invisibly returns a list with:

- `alpha_code`: Species code
- `extent`: List with `ul` and `lr` coordinate pairs
- `paths`: List with paths to `extent.txt` and log file
- `notes`: Character vector of orientation warnings or NULL

Examples

```
## Not run:
  set_extent("CASP")
  set_extent("AMRO", ul = c(-130, 55), lr = c(-60, 20))

## End(Not run)
```

set_up_run *Initialize an rENM run directory structure*

Description

Create the directory layout required for a single rENM modeling run identified by a four-letter species alpha code. Missing directories are created while existing directories are preserved.

Usage

```
set_up_run(alpha_code, project_dir = NULL)
```

Arguments

alpha_code	Character(1). Four-letter species alpha code (e.g., "CASP").
project_dir	Character(1) or NULL. Optional project root directory. If NULL, resolved via rENM_project_dir().

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Inputs: The base project directory is resolved using:

rENM_project_dir()

The run directory is constructed as:

<project_dir>/runs/<alpha_code>

Directory structure: The following directories are created if they do not already exist:

- _occs/ occurrence input/output files
- _vars/ cropped predictor rasters
- Trends/ trend analysis outputs
- Summaries/ plots, tables, and reports
- TimeSeries/<year>/model/ model outputs
- TimeSeries/<year>/occs/ processed occurrences
- TimeSeries/<year>/vars/ environmental predictors

Additional subdirectories under Trends/ include:

- centroids/
- suitability/
- variables/

Time series directories are created for 5-year bins spanning 1980 to 2020.

Outputs: A standardized log file is created or appended at:

<project_dir>/runs/<alpha_code>/ _log.txt

recording created and existing directories.

Methods:

- Resolves project directory
- Constructs run-specific directory paths
- Creates missing directories recursively
- Records created and pre-existing paths
- Appends a processing summary to the run log

Data requirements: No input data are required. This function initializes structure only and does not validate or process data files.

Value

A list with:

- `alpha_code`: Species alpha code
- `run_dir`: Full path to the run directory
- `created`: Character vector of newly created directories
- `existing`: Character vector of directories that already existed
- `log_file`: Path to the run log file

Examples

```
## Not run:
  Sys.setenv(RENM_PROJECT_DIR = "~/rENM")
  run <- set_up_run("CASP")

  run <- set_up_run("CASP", project_dir = "/projects/rENM")

  run$run_dir
  run$created

## End(Not run)
```

---

thin_occurrences *Apply spatial thinning to occurrence records (sequential version)*

Description

Applies spatial thinning to occurrence records to reduce spatial clustering by enforcing a minimum distance between retained points. This helps mitigate spatial sampling bias in ecological niche models.

Usage

```
thin_occurrences(alpha_code, thin_distance, project_dir = NULL)
```

Arguments

alpha_code	Character. Four-letter species alpha code (e.g., "CASP").
thin_distance	Numeric. Minimum allowed distance (in kilometers) between retained occurrence points.
project_dir	Character. Optional rENM project root directory. If NULL, the directory is resolved using rENM_project_dir via: 1. Explicit project_dir argument 2. options(rENM.project_dir) 3. RENM_PROJECT_DIR environment variable Providing project_dir explicitly is recommended for reproducible scripts and package testing.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Pipeline context Spatial thinning is typically applied after duplicate removal and prior to model fitting. The thinning algorithm enforces a minimum separation distance between retained occurrence points, reducing over-representation of heavily sampled locations.

The rENM project directory is resolved using rENM_project_dir, ensuring that all filesystem operations are portable and do not rely on hard-coded paths such as ~/rENM.

Inputs This function operates on per-bin occurrence CSV files located in:

<rENM_project_dir>/runs/<alpha_code>/_occs/tmp/

Files must follow the naming pattern:

of-<year>.csv

Each file must contain columns:

- longitude
- latitude

Outputs Thinned datasets are written back to the same directory:

<rENM_project_dir>/runs/<alpha_code>/_occs/tmp/

Files are overwritten in place.

Methods A sequential thinning algorithm is applied:

- Records are processed in input order
- The first point is always retained
- Each subsequent point is compared to all retained points
- Distances are computed using the Haversine formula
- Points closer than the specified threshold are removed

The Haversine distance is computed in kilometers using a spherical Earth approximation.

Data requirements The temporary occurrence directory must exist and contain one or more CSV files matching the expected naming pattern and column structure.

Centralizing project-directory resolution via rENM_project_dir ensures that all functions in the rENM package operate relative to a single configurable project root, improving reproducibility and enabling use in HPC or multi-project environments.

Value

List. Invisibly returns a structured list containing:

alpha_code Character. Species alpha code.

files_processed Character vector. Full paths to processed occurrence files.

points_before Named integer vector. Number of occurrence points before thinning for each file.

points_after Named integer vector. Number of occurrence points after thinning for each file.

output_dir Character. Directory containing thinned occurrence files.

log_file Character. Path to the _log.txt file updated with processing details.

Side effects:

- Overwrites occurrence CSV files with thinned datasets
- Reduces point counts according to the thinning distance
- Appends a processing summary to the log file

See Also

rENM_project_dir

Examples

```
## Not run:

# Standard workflow
thin_occurrences("CASP", thin_distance = 10)

# Reproducible script
thin_occurrences("CASP", thin_distance = 10,
                 project_dir = "/projects/rENM")


## End(Not run)
```

`thin_occurrences2` *Apply spatial thinning to occurrence records (parallel version)*

Description

Applies spatial thinning and optional record capping to per-year occurrence CSV files using parallel processing, writing results back to disk in place for efficient large-scale preprocessing.

Usage

```
thin_occurrences2(alpha_code, radius = 1, records = 250, workers = NULL)
```

Arguments

alpha_code	Character. Four-letter species alpha code used to locate the run directory.
radius	Numeric. Minimum allowed distance between retained points in kilometers. Use 0 to disable spatial thinning.
records	Integer. Maximum number of records to keep per file after thinning. Use 0 to keep all available records.
workers	Integer, NULL. Number of parallel workers to use. If NULL, uses all available logical cores.

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Pipeline context This function is the parallel counterpart to the serial thinning routine and is intended for larger species datasets where per-year files are numerous or large.

Spatial thinning is typically applied after duplicate removal and prior to model fitting, reducing spatial clustering and sampling bias.

Inputs For a given species alpha code, the function looks for files named:

`of-YYYY.csv`

in:

`rENM_project_dir()/runs/<alpha_code>/_occs/tmp/`

Each file is expected to contain at least:

- species
- longitude
- latitude

Files with zero rows or missing coordinate columns are skipped.

Outputs Processed files are written back in place to:

`rENM_project_dir()/runs/<alpha_code>/_occs/tmp/`

Files are overwritten with thinned and optionally capped datasets.

Methods Spatial thinning (radius)

If radius > 0, a greedy nearest-neighbor filter is applied:

- The first point is always retained
- Each subsequent point is retained only if it is at least radius kilometers away from all previously retained points
- Distances are computed using the Haversine formula
- Earth radius is assumed to be 6371 km

The number of records surviving this step is reported as kept_after_radius.

Record cap

If records > 0, the number of rows is further capped to at most records per file using random downsampling. The final row count is stored in kept_after_records, and the after column reports the final number of rows written back to disk.

Parallel execution

Parallelism is handled by the future and future.apply packages using a multisession plan:

- If workers is NULL, the number of workers defaults to the number of available logical cores (at least 1)
- Each file is processed independently on a worker
- Files are overwritten in place after processing

Both future and future.apply must be installed and loadable; if not, the function stops with an informative error.

Logging

A standardized processing summary is appended to:

`rENM_project_dir()/runs/<alpha_code>/_log.txt`

under the section title:

"Processing summary (thin_occurrences2 parallel)"

The log entry includes:

- number of files processed
- total rows before and after thinning
- total rows removed
- number of files changed
- thinning radius and records cap used

Centralizing project-directory resolution via `rENM_project_dir` ensures CRAN-compliant, portable behavior across systems.

Value

Data frame. Invisibly returns a data frame with one row per processed file and columns:

file Character. File name (e.g., "of-2015.csv").

before Integer. Number of rows before thinning.

kept_after_radius Integer. Rows retained after spatial thinning, or NA if disabled or skipped.

kept_after_records Integer. Rows retained after applying the records cap, or NA if no cap was applied.

after Integer. Number of rows written back to disk.

Side effects:

- Overwrites occurrence CSV files with processed datasets
- Applies spatial thinning and optional record capping
- Executes processing in parallel across worker processes
- Appends a processing summary to the log file

Examples

```
## Not run:
out <- thin_occurrences2(
  alpha_code = "CASP",
  radius     = 10,
  records    = 500,
  workers    = NULL
)

out

## End(Not run)
```

`tidy_occurrences` *Finalize occurrence files by moving from temporary to main directory*

Description

Moves per-bin occurrence CSV files from a temporary staging directory into the main occurrence directory for a species run, overwriting existing files and removing the temporary directory upon completion.

Usage

```
tidy_occurrences(alpha_code, project_dir = NULL)
```

Arguments

`alpha_code`	Character. Four-letter species alpha code (e.g., "CASP").
`project_dir`	Character. Optional rENM project root directory. If NULL, the directory is resolved using `rENM_project_dir` via: 1. Explicit project_dir argument 2. options(rENM.project_dir) 3. RENM_PROJECT_DIR environment variable

Details

This function is part of the rENM framework's processing pipeline and operates within the project directory structure defined by rENM_project_dir().

Pipeline context This function is typically the final cleanup step after:

- occurrence retrieval
- duplicate removal
- spatial thinning

The rENM project directory is resolved using `rENM_project_dir`, ensuring that all filesystem operations are portable and do not rely on hard-coded paths such as ~/rENM.

Inputs Occurrence files are expected in:

`<project_root>/runs/<alpha_code>/_occs/tmp/`

Files must be CSV format with names corresponding to time bins.

Outputs Files are copied into:

`<project_root>/runs/<alpha_code>/_occs/`

Existing files with the same name are overwritten.

Methods This function performs a destructive finalization step:

- Files in _occs/tmp/ are copied into _occs/
- Existing files in _occs/ with the same name are overwritten
- The _occs/tmp/ directory is deleted after successful transfer

This step signals that occurrence preprocessing is complete and that the resulting files are ready for downstream modeling.

Data requirements The temporary directory must exist and contain at least one CSV file.

Centralizing project-directory resolution via `rENM_project_dir` ensures consistent behavior across all rENM functions and supports reproducibility, portability, and HPC use.

Value

List. Invisibly returns a structured list containing:

alpha_code Character. Species alpha code.

files_moved Character vector. Full paths to files moved into the destination directory.

destination_dir Character. Path to the final _occs/ directory.

log_file Character. Path to the _log.txt file updated with processing details.

Side effects:

- Copies CSV files from the temporary directory into the main occurrence directory
- Overwrites existing files with matching names
- Deletes the temporary directory after transfer
- Appends a processing summary to the log file

See Also

rENM_project_dir

Examples

```
## Not run:

tidy_occurrences("CASP")

tidy_occurrences("CASP", project_dir = "/projects/rENM")


## End(Not run)
```

Index